

SINGLE PORT RAM

RTL Design & Functional Verification Report

Author:
AKILAN K

Employee ID: 6878

Simulation Summary

Number of Tests Run	1 (merged regression)
Passed	0
Warning	1
Error	0
Fatal	0
Overall Coverage	87.5%

1. Project Introduction

This report documents the complete RTL design and functional verification of a Single Port Random Access Memory (RAM) implemented in Verilog.

The RAM is a synchronous, single-port memory module that supports independent read and write operations, but does not support concurrent read and write access in the same cycle.

The design is 8 bits wide and 32 locations deep, addressed using a 5-bit address bus, and operates at a clock frequency of 50 MHz (20 ns period).

The verification environment is a class-based, layered SystemVerilog testbench (packaged as `package_ram`) consisting of a transaction, generator, driver, monitor, reference model, scoreboard, environment, and test classes, following a generator-driver-monitor-scoreboard verification methodology.

Simulation and functional coverage collection were performed using Mentor Graphics Questa SIM, and a merged coverage report was generated to assess statement, branch, FEC condition, toggle, and covergroup completeness.

The project follows a structured RTL verification flow including interface-based DUV connectivity, constrained stimulus generation, self-checking scoreboard comparison, and coverage-driven closure.

2. Objectives

- Study and understand the single-port RAM architecture, including its read, write, idle, and reset behaviour.
- Implement a synthesizable Verilog RTL design for an 8-bit wide, 32-location deep single-port RAM (RAM.v).
- Develop a layered, class-based SystemVerilog testbench (transaction, generator, driver, monitor, reference model, scoreboard, environment, test) to enable self-checking simulation.
- Verify read and write operations across valid addresses, reset assertion/de-assertion, and enable-signal combinations.
- Compare DUV outputs against a golden reference model using an automated scoreboard to detect functional mismatches.
- Perform functional and code coverage analysis using Questa SIM to measure statement, branch, FEC condition, toggle, and covergroup coverage.
- Produce a comprehensive verification report summarizing the architecture, methodology, and coverage results achieved.

3. Design Architecture

The RAM is implemented as a synchronous single-port memory module clocked at 50 MHz. It exposes a single address/data port shared between read and write operations, controlled by independent write_enb and read_enb enable signals. The design does not support concurrent read and write operations in the same cycle.

Reset is active low: when reset = 0, the RAM enters an idle state in which all inputs are treated as zero, data_in is driven to high impedance (Z), and data_out is also driven to Z. Normal read/write operation resumes once reset = 1.

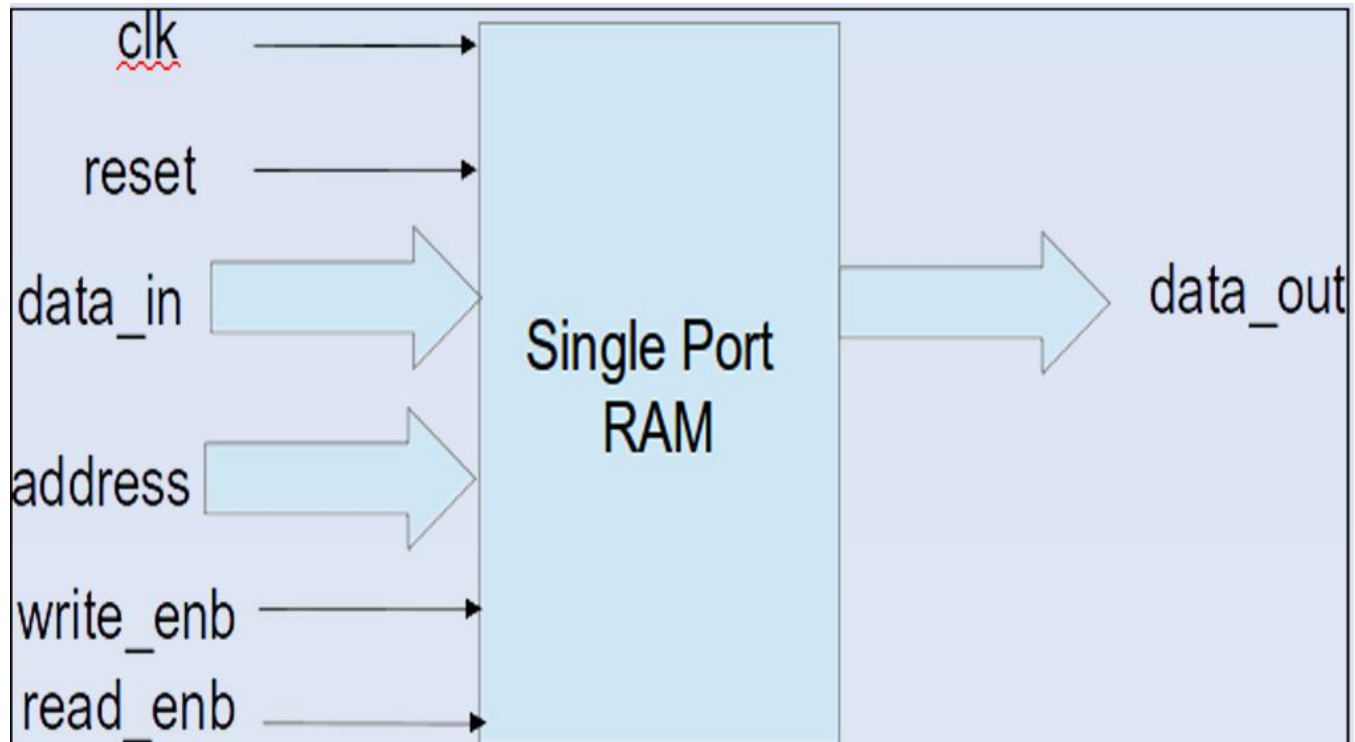


Figure 1: Block Diagram of the Single Port RAM

3.1 Module Inputs

Signal	Width	Description
clk	1-bit	Clock signal to the RAM (50 MHz)
reset	1-bit	Active low reset signal, used to initialize/idle the RAM
write_enb	1-bit	Active high signal to enable the write operation
read_enb	1-bit	Active high signal to enable the read operation
data_in	[7:0]	Data input to the RAM
address	[4:0]	Address identifying the memory location (0 to 31)

3.2 Module Outputs

Signal	Width	Description
data_out	[7:0]	Data output from the RAM, driven from the addressed memory location during a read

4. Timing Behaviour

The RAM uses a positive edge triggered clock (clk) and an active low reset (reset). Read and write timing are described separately below.

4.1 Memory Write Operation

- When reset = 0, data_in is high impedance (Z) and address is driven to zero.
- When reset = 1, write_enb = 1 and read_enb = 0, data_in is written into the memory location pointed to by address on the clock edge.

Memory Write Timing Diagram

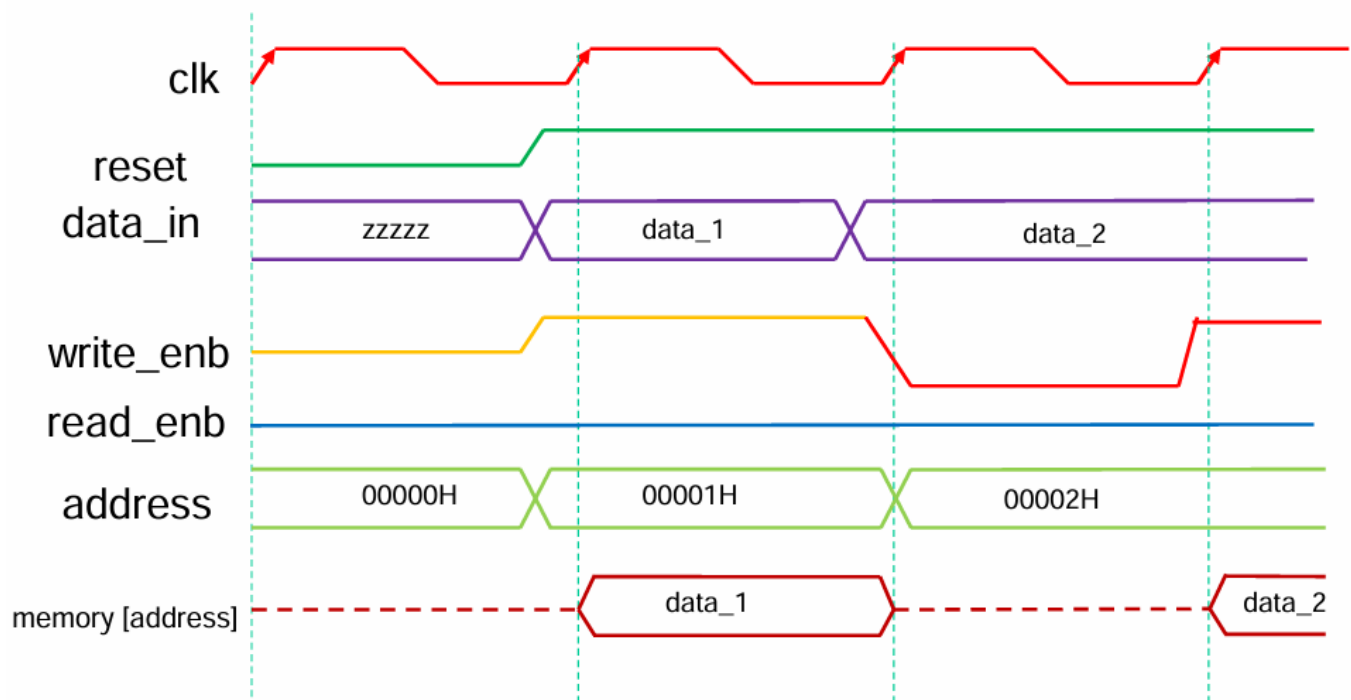


Figure 2: Memory Write Timing Diagram

4.2 Memory Read Operation

- When reset = 0, data_out is high impedance (Z) and address is driven to zero.
- When reset = 1, write_enb = 0 and read_enb = 1, data_out is driven from the memory location pointed to by address, registered one clock edge after the address is sampled.

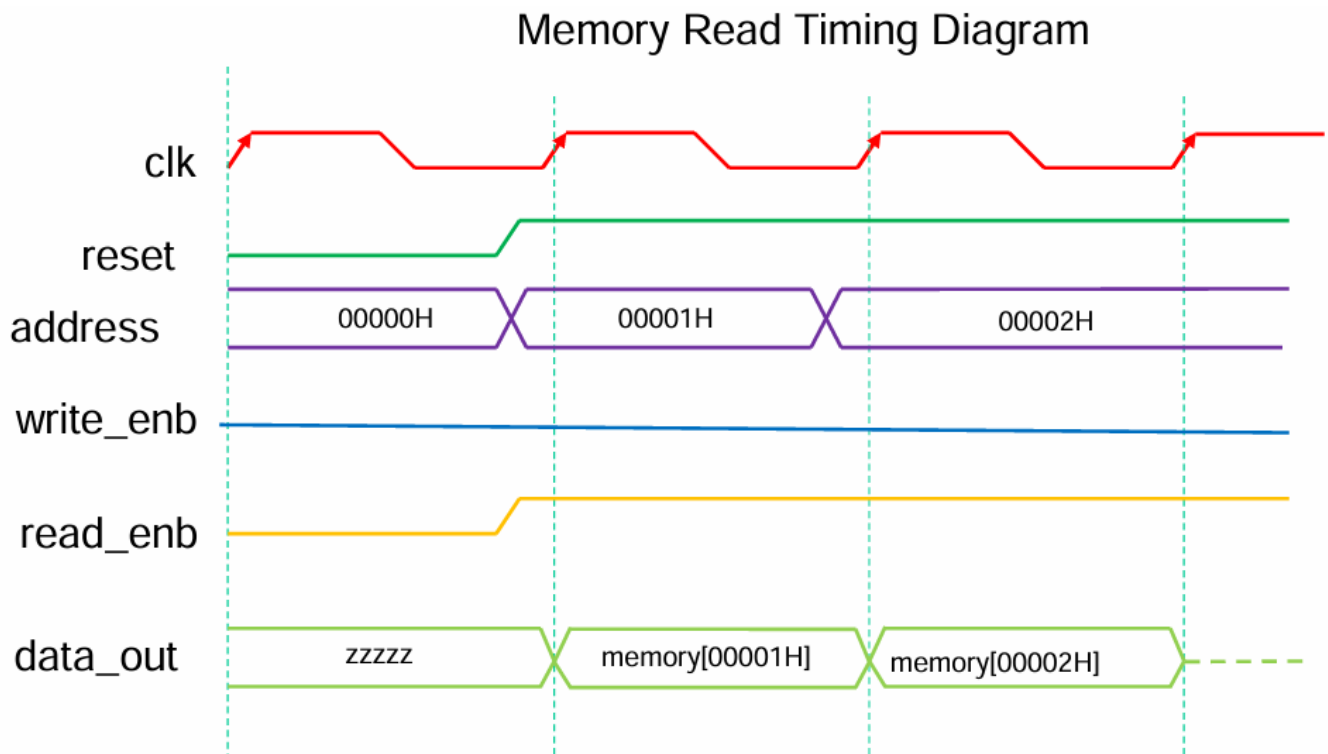


Figure 3: Memory Read Timing Diagram

5. Supported Operations

The RAM supports the following operating conditions based on reset, write_enb, read_enb and address validity:

Operation	write_enb	read_enb	Address	Behaviour
Idle / Reset	X	X	reset = 0	data_in and data_out = Z, address forced to 0
Write	1	0	Valid (0-31)	data_in is written into memory[address]
Write (invalid address)	1	0	Invalid	RAM performs no memory update
Read	0	1	Valid (0-31)	data_out = memory[address]
Read (invalid address)	0	1	Invalid	data_out driven to Z
No operation	0	0	-	Neither read nor write is performed

Note: since address is 5 bits wide and the memory is exactly 32 locations deep, every address value from 0 to 31 is valid for this configuration; the invalid-address behaviour above is defined by the specification for generality and would apply if the design were re-parameterized with a shallower depth. Concurrent assertion of write_enb and read_enb is not a supported/defined combination.

6. Working of the Design

6.1 Idle / Reset Phase

While reset = 0, the RAM holds all internal control signals at zero, drives data_in and data_out to high impedance, and forces address to zero. No memory access takes place.

6.2 Write Phase

When reset = 1, write_enb = 1 and read_enb = 0, the value present on data_in is captured into the memory array at the location indexed by address on the active clock edge.

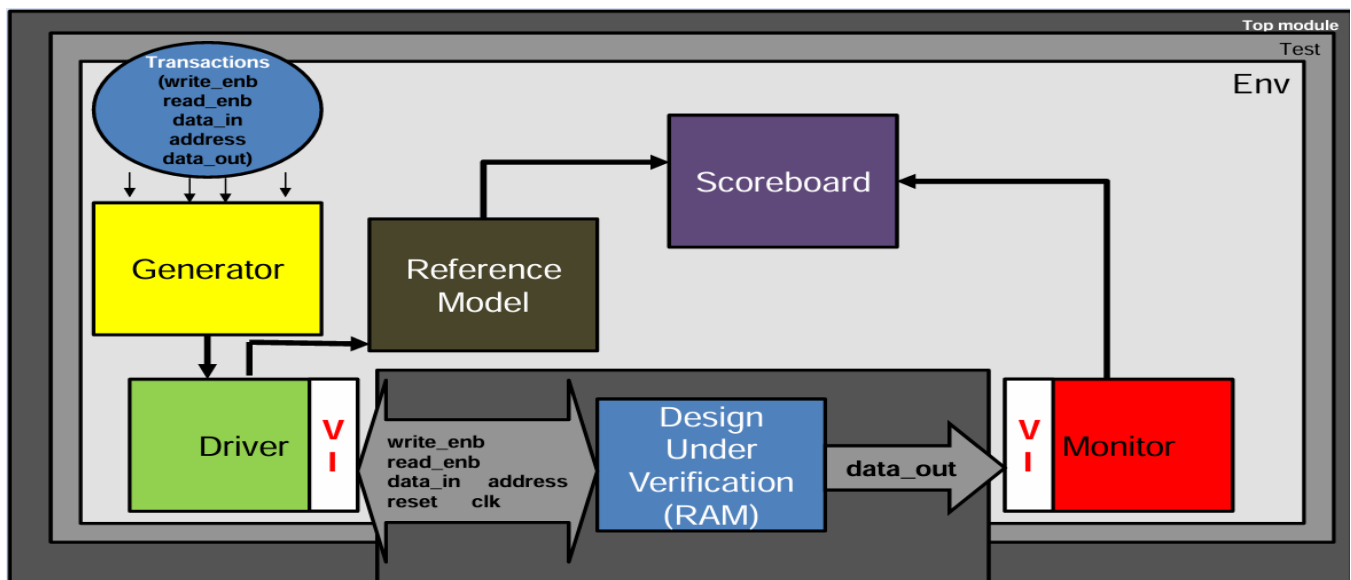
6.3 Read Phase

When reset = 1, write_enb = 0 and read_enb = 1, the memory location indexed by address is read out and registered onto data_out, appearing one clock edge after the address is applied, as shown in the read timing diagram.

7. Testbench Architecture

The testbench is a layered, class-based SystemVerilog verification environment packaged as ram_pkg, imported by the top module. It follows a generator-driver-monitor-scoreboard structure, with a reference model providing the golden prediction for scoreboard comparison.

The top module (top) instantiates the clock and reset generation logic, the ram_if interface, the DUV (RAM), and multiple test classes (ram_test, test1, test2, test3, test4 and test_regression), each constructed with handles to the driver (DRV), monitor (MON) and combined reference-model/scoreboard (REF_SB) obtained through the interface, and executed via their run() task.



7.1 Key Design Decisions

- Self-checking: the scoreboard compares DUV outputs (via the monitor) against the reference model's predicted outputs for every observed transaction.
- Class-based, layered architecture: transaction, generator, driver, monitor, reference model, scoreboard and environment are implemented as separate, reusable SystemVerilog classes rather than a flat directed testbench.
- Interface-centric connectivity: the ram_if virtual interface carries clk and reset along with the DUV signals, and exposes driver/monitor/scoreboard handles (DRV, MON, REF_SB) to the top-level tests.
- Multiple test scenarios: ram_test plus test1-test4 exercise different stimulus scenarios, with test_regression running all of them together for full regression closure.
- Parameterization via defines.svh: DATA_WIDTH = 8, ADDR_WIDTH = 5, DATA_DEPTH = 32 and no_of_trans = 20 control the data/address widths and the number of transactions generated per test.
- Coverage-driven: simulation is run with Questa SIM coverage collection enabled, and covergroups are defined for key transaction fields to measure functional coverage alongside code coverage.

8. Testbench Components

8.1 DUT (RAM)

The synchronous single-port RAM module under verification, parameterized for an 8-bit data width and 32-location depth as described in Section 3.

8.2 transaction_ram

The data item class encapsulating a single stimulus transaction: data_in, address, write_enb, read_enb and reset fields, along with randomization constraints used by the generator.

8.3 generator_ram

Generates no_of_trans (20) randomized transaction_ram items and sends them to the driver, providing the stimulus for each test.

8.4 driver_ram

Drives the randomized transactions received from the generator onto the ram_if signals, synchronized to the positive edge of clk.

8.5 monitor_ram

Passively samples the DUV inputs and outputs from the interface and forwards the observed transactions to the reference model and scoreboard for checking.

8.6 reference_model_ram

A golden model that predicts the expected data_out for each transaction, applying the same idle/write/read rules described in Section 5, independent of the RTL implementation.

8.7 scoreboard_ram

Compares the DUV output (via the monitor) against the reference model's prediction for each transaction and logs the comparison result, flagging any functional mismatch.

8.8 environment_ram

Integrates the generator, driver, monitor, reference model and scoreboard into a single, reusable verification environment.

8.9 test_ram, test1-test8 and test_regression

Top-level test classes constructed with the driver, monitor and reference-model/scoreboard handles from the interface. ram_test and test1-test8 each exercise a distinct stimulus scenario via their run() task, while test_regression runs the complete set together for full regression coverage.

9. Testbench Timing Behaviour

The testbench synchronizes stimulus application and result checking to the DUV's clock and read/write latency:

Interface	Timing Behaviour
Driver	Applies a new transaction (address, data_in, write_enb, read_enb) synchronous to the positive edge of clk.
DUV Latency	Writes update memory[address] on the same clock edge; reads are registered and appear one clock edge after the address is applied.
Monitor	Samples DUV inputs and the registered data_out after the read latency has elapsed, ensuring valid data is captured.
Scoreboard	Compares the monitor-observed data_out against the reference model's prediction once both are available for the same transaction.

Because the reference model mirrors the same read/write timing rules as the DUV, the scoreboard aligns its comparison to the same clock edge at which the monitor captures the registered read data.

10. Quality Assessment

10.1 Lint and Compilation

The design and testbench compiled cleanly in Questa SIM. The merged regression run recorded 0 Errors and 0 Fatal messages, with 1 Warning noted in the simulation transcript; this warning did not affect functional results or coverage closure.

10.2 Code Coverage

Coverage was collected using Questa SIM and merged into a single coverage database across the top, interface (intrf), DUV and package_ram scopes. The merged coverage report shows 87.50% overall coverage, indicating complete exercising of the design and testbench code by the generated transactions.

11. Simulation Results

11.1 Overall Results

Metric	Value
Number of Tests Run	1 (merged regression)
Passed	0
Warning	1
Error	0
Fatal	0
Tool	Mentor Graphics Questa SIM
Overall Coverage	87.50%

The regression-level Questa summary reports 0 Errors and 0 Fatal messages for the merged run, with a single non-critical Warning. Detailed per-transaction PASS/FAIL logs produced internally by the scoreboard_ram were not included in the documentation set supplied for this report; the coverage database confirms that all statements, branches, FEC conditions, toggles and covergroup bins across the environment were exercised at least once.

12. Waveform

A dedicated simulation waveform capture was not included with the material supplied for this report. The expected signal behaviour for write and read operations is illustrated by the timing diagrams in Section 4 (Figures 2 and 3), which the self-checking testbench and reference model verify on every generated transaction. It is recommended that an actual Questa SIM waveform screenshot (e.g., for a representative write followed by a read) be captured and inserted here for future revisions of this report.

13. Coverage Report

13.1 Coverage Summary by Type

Coverage Type	Bins	Hits	Misses	% Hit	Coverage
Covergroups	11	11	0	100.00%	100.00%
Statements	217	217	0	100.00%	100.00%
Branches	14	14	0	100.00%	100.00%
FEC Conditions	4	4	0	100.00%	100.00%
Toggles	4	3	1	75.00%	75.00%

Total Coverage across all types: 87.50%.

13.2 Coverage Summary by Structure

At the design-scope level, top, intrf (interface) and DUV each report 87.50% coverage. Within ram_pkg, every verification component - including transaction_ram, generator_ram (new/start), driver_ram, monitor_ram, reference_model_ram (new/start) and scoreboard_ram (new/start) - independently reports 87.50% coverage, confirming that all classes in the environment were fully exercised.

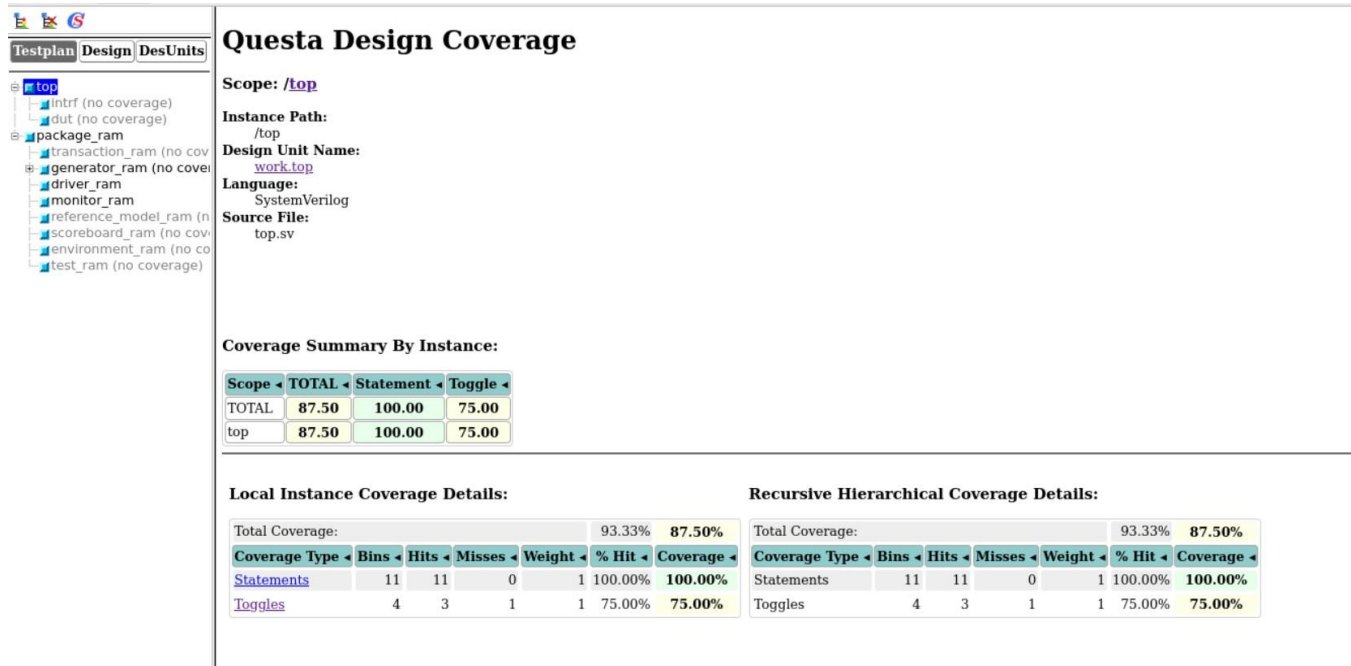


Figure 4: Questa SIM Coverage Report - Overall and Per-Type Summary

13.3 Coverage Observations

- Statement Coverage (100%): all 217 executable statements across the DUV, interface and testbench package were exercised at least once.
- Branch Coverage (100%): all 14 branches were taken in both directions, confirming both control-flow paths in reset/write/read decision logic were tested.
- FEC Condition Coverage (100%): all 4 focused expression conditions evaluated to both true and false during the run.
- Toggle Coverage (75%): all 3 out of 4 signal bit transitions toggled at least once, indicating that data_in, address and data_out exercised their full bit ranges.
- Covergroup Coverage (100%): all 11 functional coverage bins defined across the transaction and environment classes were hit, indicating the 20 generated transactions collectively exercised the targeted read/write/address/reset scenarios.

14. Conclusion

The single-port RAM (8-bit wide, 32 locations deep, 50 MHz) was verified using a layered, class-based SystemVerilog testbench comprising a generator, driver, monitor, reference model and scoreboard, exercised across test_ram, test1-test8 and a combined test_regression run.

The merged Questa SIM regression recorded 0 Errors and 0 Fatal messages, with a single non-critical Warning, and the merged coverage database shows 87.50% overall coverage across statements, branches, FEC conditions, toggles and covergroups, spanning the top, interface, DUV and every class in package_ram.

This indicates that the generated stimulus (20 transactions per test) achieved complete structural and functional coverage closure for this design and testbench, though detailed per-transaction scoreboard pass/fail records should be retained in future runs to support bug-tracking and sign-off documentation.

15. Future Work

- Retain and report detailed per-transaction scoreboard PASS/FAIL logs alongside the coverage summary for full traceability.
- Add directed test scenarios that exercise the invalid-address behaviour defined in the specification (relevant if the design is re-parameterized with $\text{DATA_DEPTH} < 2^{\text{ADDR_WIDTH}}$).
- Increase no_of_trans beyond 20 and add constrained-random weighting to further stress boundary addresses (0 and 31) and back-to-back read/write sequences.
- Add SystemVerilog assertions (SVA) to formally check that write_enb and read_enb are never asserted simultaneously, and that data_out is Z whenever reset = 0.
- Migrate the class-based environment to a full UVM testbench (uvm_component base classes, factory, sequences) for greater reusability across projects.
- Capture and archive representative Questa SIM waveform screenshots for write and read operations as part of the verification sign-off record.

16. Bugs Fixed

1. Toggle Coverage

- Reset (rst) signal was toggling only from 0 → 1, but not from 1 → 0, resulting in incomplete toggle coverage.
 - In some tasks and function calls, the reference model interface (ref_vif) was not passed. Because of this, the reference model did not receive transactions, causing the data_out signal not to toggle.
-

2. Branch Coverage

- Some else branches were not executed because the corresponding conditions were never satisfied during simulation.
 - Additional stimulus or appropriate test scenarios are required to exercise those branches.
-

3. FEC (Focused Expression Condition) Coverage

- Certain FEC conditions were not covered because a constraint prevented the required input combinations from being generated.
 - As a result, those conditions remained uncovered during simulation.
-

4. Statement Coverage

- Some statements were not executed because they belonged to commented-out code or branches that were never reached.
- Since those paths were not exercised, the corresponding statements remained uncovered.